

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

ORGANISATION OF ISLAMIC COOPERATION (OIC)

Department of Computer Science and Engineering(CSE)

Final Project Report

Project Name

Dengue Alert

Submitted By:

Group 14

Md Rufayed Ul Alam (230042121)

Sheikh Irtiaj Ahmed (230042130)

Adiba Ahsan Raiyan (230042161)

SWE 4404 : SPL-II

Supervised By:

Ajwad Abrar Mostofa,

Junior Lecturer

Department of Computer Science and Engineering

Table Of Contents

1. Executive Summary .....……**03**
2. Problem Domain and Background .....……**04**
3. Literature and Existing Solutions .....……**06**
4. Requirements (SRS Session) .....……**08**
5. System Design (Design Session) .....……..**12**
6. Implementation (Implementation Session) .....……..**16**
7. Tools, Version Control, Testing, and CI/CD .....……..**20**
8. Professional, Ethical, and Legal Considerations .....……**23**
9. Project Management and Feasibility .....……..**25**
10. Teamwork and Individual Contributions .....……..**28**
11. Results, Limitations, and Future Work .....……**29**
12. Glossary and References .....……..**31**

1. Executive Summary

The **Dengue Alert & Ward-Based Management System** is a mobile-first web application designed to help residents report possible mosquito-breeding locations and help local authorities coordinate cleanup work. In a typical manual process, a resident may notice standing water or an uncovered container but may not know which local office to contact or whether anyone acted on the complaint. The project creates a traceable route from a citizen report to a field response.

A citizen can create an account using email and password, verify the email address, or use Google sign-in. The reporting screen captures a live camera image and the device's GPS coordinates. The server checks the location against ward boundary polygons in PostgreSQL/PostGIS and stores the report with its image in Cloudinary. A Councillor can review reports for the assigned ward and assign an Inspector. The Inspector uploads a cleanup photograph and GPS coordinates. The Councillor then approves the evidence or sends the work back for reassignment. The citizen can view the current report status.

The system also detects complaint clusters. In the supplied backend, the default hotspot rule is **five or more complaints within 200 metres**, evaluated within a ward. When a new hotspot is created, the backend can email a time-limited, read-only report link to the configured Magistrate email address. Citizens can opt in to browser-based proximity alerts for active hotspots. The highest application role is named **Master Admin**; the field-response role is named **Inspector**.

The latest code delivers an integrated workflow for citizen reporting, GIS-based ward routing, Inspector assignment, photo-and-GPS proof submission, Councillor verification, staff management, and hotspot escalation. Together, these capabilities form a working end-to-end prototype that supports the journey from report submission to verified cleanup. The current implementation provides a solid foundation for further enhancements, including offline reporting, password recovery, duplicate detection and priority management, task-start and work-history tracking, automated email retries, ward and threshold configuration, audit and status-history records, automated testing, CI/CD, and deployment documentation.

For a non-technical reader, the outcome is a practical prototype that connects community observations to a local cleanup process and records photographic and location evidence. It does not diagnose dengue, measure mosquito density, or prove that an area is epidemiologically high-risk. A hotspot is a configurable cluster of submitted complaints and should be interpreted as an operational alert, not a clinical or public-health diagnosis.

CO6 response. The project can be explained without technical jargon as a reporting and follow-up service: residents submit evidence, the correct ward receives the case, field staff upload proof, and a Councillor confirms the result. The report distinguishes implemented capabilities from planned work so that a reader can understand the actual outcome.

2. Problem Domain and Background

2.1 Application Domain

The application integrates dengue prevention, community reporting, local-government service delivery, and geographic information systems. WHO guidance identifies source reduction—eliminating mosquito breeding sites—as a key preventive strategy and emphasizes community participation and local knowledge in sustainable vector control [1][2]. WHO's dengue risk-communication toolkit further highlights the importance of collecting context-specific information and using evidence to guide prevention and response planning [3].

The project addresses these needs through a digital reporting and coordination platform. Rather than replacing trained vector-control personnel, laboratory surveillance, clinical diagnosis, or official outbreak reporting, it complements these activities by providing residents and local authorities with a structured approach to documenting suspected breeding sites and coordinating follow-up actions.

2.2 Problem Statement

Residents may observe standing water, uncovered containers, blocked drains, or discarded objects that retain water. A report can be delayed or lost when the resident does not know the responsible ward office, when a report has no precise location, or when the cleanup process has no shared evidence trail. Authorities may also have difficulty identifying clusters of complaints and escalating them to senior officials.

The project addresses four connected problems:

1. **Intake:** make it straightforward to submit a photo and precise GPS location.
2. **Routing:** identify the responsible ward from the geographic point rather than relying on a manually typed address.
3. **Accountability:** assign the work to a named Inspector and require an after-cleanup photo with GPS.
4. **Awareness:** identify clusters of complaints and notify the configured Magistrate and nearby opted-in users.

2.3 Stakeholders and Users

Stakeholder / Actual Role	Need and Responsibility
Citizen	Browse the public overview, register or sign in, submit a suspected breeding-site report, and view its current status. Citizens may opt in to location-based hotspot alerts.
Councillor	Review reports associated with the Councillor's ward, assign an Inspector, compare before/after evidence, resolve or reassign work, and reject invalid reports.
Inspector	View assigned reports, inspect the reported location and before photo, and upload an after-cleanup photo with GPS coordinates and an optional comment.
Master Admin	View reports across the system, create and deactivate Councillor and Inspector accounts, review staff by ward, view active hotspots, and manually resend a Magistrate alert.
Zonal Executive Magistrate	Open a time-limited, read-only hotspot report using the emailed token link; no application account is required.
System Operator / Developer	Configure database, Cloudinary, SMTP, Google Identity, environment variables, ward data, and deployment.

2.4 Fundamental Software Concepts Used (CO1)

- **Programming and Control Flow:** React components manage UI state and events; Express route handlers validate requests and select actions according to role and report status.
- **Data Structures:** Arrays hold report and staff lists; objects represent users, reports, and API responses; `Map` and `Set` are used for report merging and hotspot tracking; `FormData` carries image uploads.
- **Database Concepts:** Relational tables represent users, wards, reports, hotspots, and alert links. Foreign-key relationships are implied by joins and error handling.
- **Transactions and Consistency:** Report creation uses a PostgreSQL transaction, locks the user's row while enforcing the rolling 24-hour quota, and uses a savepoint so a hotspot-detection failure does not automatically discard an otherwise valid report.
- **Spatial Data:** A GPS point is matched to a ward polygon using `ST_Covers`; distance-based checks use PostGIS geography operations such as `ST_DWithin` and `ST_Distance`.
- **Software Design:** The frontend is divided into pages, reusable components, context providers, and API configuration. The backend is divided into routes, middleware, configuration, and service modules.

2.5 Specialist Knowledge Required

1. **Dengue Prevention and Source Reduction:** The system records suspected environmental breeding sites. Reports is reviewed by people who understand the difference between a suspected hazard and a confirmed public-health finding.
2. **GIS and Spatial Databases:** Ward routing requires valid polygon boundaries, a consistent coordinate reference system, and correct latitude/longitude ordering. Proximity alerts require distance calculations in metres.
3. **Municipal Workflow Design:** Report intake, assignment, field evidence, verification, rejection, and escalation is represented as controlled state transitions.
4. **Authentication and Access Control:** Citizens must see their own reports; Inspectors must see assigned work; Councillors must be restricted to their ward; Master Admin has broader oversight.
5. **Privacy and Evidence Handling:** Precise locations, photographs, and contact details can reveal sensitive information. Access, retention, and sharing must be controlled.

2.6 Scope Boundary and Public-Health Caution

The implementation identifies complaint clusters by counting submitted reports within a defined area. It does not incorporate entomological survey findings, laboratory-confirmed dengue cases, mosquito-density measurements, or clinical records. Accordingly, “hotspot” refers exclusively to an area that meets the application's configured complaint-cluster threshold, rather than a scientifically confirmed dengue-transmission hotspot. These thresholds should be reviewed and validated by public-health specialists and relevant city authorities before real-world deployment.

CO1 response: Developing this system requires expertise in programming, data structures, relational database design, transaction management, access control, and spatial computation. It also requires specialist knowledge of dengue source reduction, GIS, municipal workflows, and privacy, as the application transforms location-based public reports into actionable tasks for local authorities.

3. Literature and Existing Solutions

The original SRS contains a glossary and requirements but does not provide a formal literature review or a reference list. This section adds a concise review of credible public-health guidance and established digital platforms. These sources inform the project context; they do not imply that the project has been clinically validated.

3.1 WHO Dengue Prevention and Community Engagement

WHO describes community participation and local knowledge as important to sustainable dengue vector control, including the removal of mosquito egg-laying sites [1][2]. Its 2024 risk-communication and community-engagement toolkit provides practical approaches for collecting context-specific information and supporting dengue readiness and response [3]. This supports the project's citizen-reporting rationale: residents can identify local environmental concerns and provide location-specific information to the responsible authority.

Difference: WHO guidance is a public-health framework, not a software product or a municipal complaint-management workflow. The project operationalizes a narrow part of the guidance through photo/GPS reports, ward routing, cleanup assignment, and evidence review.

3.2 Open311 GeoReport v2

Open311 GeoReport v2 defines an API pattern for location-based, non-emergency service requests handled by government entities. It supports creating requests, retrieving requests, and tracking their current status [4]. This is closely related to the project's citizen-to-city reporting model.

Difference: Open311 is a general civic-service interface. The project adds dengue-oriented evidence capture, ward-boundary routing, Inspector assignment, before/after verification, complaint-cluster detection, and a time-limited Magistrate report. The supplied backend does not implement the Open311 GeoReport v2 standard; it uses its own Express REST endpoints.

3.3 Ushahidi

Ushahidi provides a crowd-sourcing and mapping platform for collecting, managing, analysing, and acting on community reports. Its documented capabilities include multiple reporting channels, configurable workflows, collaboration with roles and permissions, alerts, and mapping [5].

Difference: Ushahidi is a general-purpose platform that can be configured for different deployments. The Dengue Alert application implements a narrower workflow tailored to ward-level cleanup: a Councillor assigns an Inspector, the Inspector submits cleanup evidence, and the Councillor verifies the result. It is not a replacement for Ushahidi's broader data-gathering and analysis features.

3.4 KoboToolbox

KoboToolbox supports GPS data collection in web forms and KoboCollect, including point, line, and area capture. Its documentation also describes offline collection and queued submissions that upload when connectivity returns [6]. These capabilities are relevant to field reporting in areas with unreliable mobile connectivity.

Difference: KoboToolbox is primarily a configurable data-collection platform. The supplied project has a custom authority workflow and browser camera/GPS capture, but its service worker caches application assets rather than implementing an offline report queue. The SRS's offline-submission requirement is therefore not complete in the current source.

3.5 DHIS2 Tracker

DHIS2 Tracker supports individual-level longitudinal data and event capture for programmes such as health, education, environment, and logistics. Its implementation guidance discusses data models, roles, analytics, privacy, infrastructure, and planning for scale [7].

Difference: DHIS2 is a configurable information-system platform with broad programme and reporting capabilities. The project is a smaller, purpose-built complaint and field-response application. It does not provide DHIS2's programme configuration, national health-information integration, or mature analytics.

3.6 Technical Basis: PostGIS

The implementation uses `ST_Covers` to determine whether a ward polygon covers a submitted GPS point. PostGIS documents that `ST_Covers` includes points on the polygon boundary and can use available spatial indexes [8]. The hotspot and proximity queries use `ST_DWithin`; when geometries are cast to `geography`, the distance argument is measured in metres [9]. These functions are appropriate for ward assignment and metre-based proximity checks when the underlying geometries are valid and correctly prepared.

3.7 Synthesis and Project Contribution

Existing systems demonstrate the value of community reporting, location-aware service requests, field data capture, and workflow-based review. The project's contribution is the integration of these ideas into a focused ward-level dengue-hazard workflow:

- Live photo and GPS capture for citizen reports;
- Automatic ward assignment using PostGIS;
- Role-restricted Councillor and Inspector work queues;
- Cleanup proof with a second GPS location;
- Councillor verification and reassignment;
- Complaint-cluster detection and a secure Magistrate email link;
- Browser-based proximity alerts.

The current implementation provides a dengue-focused workflow that extends beyond a generic reporting form or civic-issue API. However, compared with a comprehensive field-data platform, it has more limited capabilities in offline operation, reporting, data governance, and deployment management.

CO2 response: The project builds upon established community-reporting and location-based service-request approaches. Its distinctive contribution lies in addressing the complete dengue-hazard response cycle, from location-based reporting and task assignment to field evidence collection and verification, rather than merely collecting location-tagged complaints. The SRS is updated to incorporate the cited sources and clearly distinguish implemented capabilities from proposed features.

4. Requirements (SRS Session)

4.1 Requirements Baseline

The SRS defines six epics: Authentication and profile management; citizen complaint reporting; geofenced routing and Councillor management; Inspector task execution; hotspot detection and alerts; and system administration/reporting. It also defines performance, security, reliability, usability, privacy, maintainability, portability, and data-integrity requirements (SRS, pp. 9–17).[10]

The following traceability uses the current code as the implementation baseline. “Partial” means that a related capability exists but does not satisfy the full SRS acceptance criteria. “Not implemented” means no supporting implementation was found in the supplied source archive.

4.2 Functional Requirement Traceability

ID	Requirement / stakeholder need	Source evidence	Current status
FR-01	Public landing page explains the service before login (Citizen).	src/pages/ LandingPage.jsx; / route in src/App.js.	Implemented
FR-02	Citizen registration and sign-in (Citizen).	userRoutes.js; AuthContext.js; Login/Register pages. Email verification and Google sign-in are present.	Partial: password reset is absent.
FR-03	Role-based access for Citizen, Inspector, Councillor, Master Admin (all users).	RoleGuard in src/App.js; authmiddleware.js; rolemiddleware.js.	Implemented
FR-04	Live photo and automatic GPS reporting (Citizen).	LiveCameraCapture.jsx; ReportIssue.jsx; POST /api/reports.	Partial: description is mandatory; guest draft is not implemented.
FR-05	Maximum three reports per user in a rolling 24-hour period (system/citizen).	routes/reportRoutes.js, report-creation transaction and quota query. Rejected reports are included in the rolling count.	Implemented
FR-06	Automatic ward routing from GPS (system/Councillor).	findWard uses ST_Covers in routes/reportRoutes.js.	Partial: unmatched points receive HTTP 422; they are not placed in a Master Admin review queue.
FR-07	Councillor sees only the assigned ward's reports (Councillor).	GET /api/reports/councillor; ward ownership condition in SQL.	Implemented
FR-08	Councillor assigns an Inspector.	PATCH /api/reports/:id/assign; InspectorAssignmentCard.jsx.	Implemented
FR-09	Inspector submits after-cleanup photo and GPS (Inspector).	CleanupUpload.jsx; PATCH /api/reports/:id.	Partial: no Start Work/In Progress step and no offline upload

			queue.
FR-10	Councillor compares before/after evidence and approves or reassigns (Councillor).	BeforeAfterComparison.jsx; PATCH /api/reports/:id/verify.	Implemented with actual statuses Resolved and Re-assigned.
FR-11	Councillor rejects an invalid report with a reason (Councillor).	PATCH /api/reports/:id/reject; ComplaintDetails.jsx.	Implemented
FR-12	Detect complaint clusters (system/Master Admin).	services/ hotspotService.js; hotspot_areas queries.	Implemented; default is five complaints within 200 m, configurable through environment variables.
FR-13	Email a 48-hour read-only Magistrate link (Magistrate).	magistrateAlertService.js; magistrateAlertRoutes.js; MagistrateHotspotDetails.jsx.	Implemented
FR-14	Warn citizens near an active hotspot (Citizen).	HotspotProximityMonitor.jsx; GET /api/hotspots/nearby.	Partial: opt-in, permission-dependent, foreground browser monitoring; no server push subscription/background tracking.
FR-15	Bangla and English UI (all users).	LanguageProvider.jsx; bn.json.	Partial: translation is dictionary-driven; full coverage has not been done.
FR-16	Queue reports and worker actions while offline (Citizen/Inspector).	Service worker caches the app shell and selected images.	Not implemented for report/action submission.
FR-17	Group duplicates and prioritize urgent/old reports (Councillor).	No duplicate-grouping or priority algorithm found.	Not implemented
FR-18	Master Admin edits wards, zones, thresholds, and Magistrate recipients (Master Admin).	Admin dashboard supports staff management and alert resend; thresholds are environment-configured.	Partial: no ward/zone/threshold management UI.
FR-19	Audit log and status-history records (system/admin/citizen).	No audit-log or status-history route/table use found.	Not implemented
FR-20	Password reset and profile-name update (user).	No reset or profile-update endpoint/page found.	Not implemented
FR-21	Inspector reminders and completed-task history (Inspector).	Inspector page shows assigned and pending-verification tasks.	Implemented
FR-22	Status-change emails and task reminders	SMTP is used for email verification and	Not implemented

	(Citizen/Councillor/Inspector).	Magistrate escalation; no report-status or delayed-task email sender was found.	
FR-23	City-wide reporting and analytics (Master Admin).	Master Admin dashboard shows counts, recent reports, staff, and hotspots.	Partial: no full analytics/export or resolution-time reporting.

4.3 Actual Role and Status Terminology

SRS Term	Actual Implementation Term / Behaviour
Super Admin / System Analyst	Master Admin
Worker / Conservancy Inspector	Inspector
Submitted / Under Review	Pending
Assigned	Assigned
In Progress	No separate implemented status or Start Work endpoint
Awaiting Verification	Pending Verification
Closed	Resolved
Rejected	Rejected
Additional implementation state	Re-assigned

The actual transition is generally `Pending → Assigned → Pending Verification → Resolved`. A rejected proof can return the report to `Re-assigned`, after which it can be assigned again. A Councillor can reject an unassigned `Pending` or `Re-assigned` report.

4.4 Non-Functional Requirement Assessment

Area	SRS Expectation	Evidence-Based Assessment
Performance	API/page response within 3 seconds; ward match within 2 seconds.	No benchmark or load-test report was supplied. Performance targets remain unverified.
Security	Password hashing, server-side RBAC, HTTPS, token expiry, upload validation, rate limiting, audit logs.	Password hashing, JWT validation, RBAC, parameterized SQL, token hashing, and some security headers exist. HTTPS enforcement, rate limiting, complete file validation, and audit logging are not evidenced.
Reliability	24/7 availability, backups, offline queue, retries.	Database transactions and cleanup logic exist. No backup procedure, availability evidence, offline queue, or automatic email retry was supplied.
Usability	Bangla/English, mobile-friendly, short reporting flow, clear errors.	Responsive pages, language switcher, live capture, these features exist. The report description is required and the complete four-step target has been measured.
Privacy	Minimize personal data; protect photos and location.	Camera capture avoids a gallery picker; location is permission-based; proximity query does not persist the submitted coordinates. A privacy

		notice, retention schedule, and deletion process are not supplied.
Maintainability	Modular routes/services and replaceable external services.	Modules are separated, but some page components and route files are large.
Portability	Major browsers and screen sizes.	PWA/service-worker support and responsive UI are present. Cross-browser testing is done practically.
Data integrity	Preserve evidence, track status changes, prevent closure without proof.	Report creation uses transactions; verification requires Pending Verification. A separate status-history/audit record is not implemented.

4.5 Documented Proposal Changes and Feedback

The SRS's Section 1.5 records four changes from the original proposal (SRS, p. 4).[10] These are the feedback-related decisions that can be verified from the supplied document.

Area	Original proposal	SRS decision	Current code alignment
Registration	Name plus phone number or email	Email-based registration	Email/password registration and Google sign-in for Citizens; staff accounts are created by Master Admin.
Media storage	Firebase Cloud Storage	Cloudinary	Cloudinary is used for before/after images.
Magistrate escalation	SMS through Twilio / SSL Wireless	Email link valid for 48 hours	Nodemailer SMTP email and 48-hour token link are implemented.
Complaint limit	Unlimited	Three per user per 24 hours	Enforced in the backend using a rolling 24-hour count.

4.6 Contextual and Sustainable-Development Considerations

The application promotes community participation, improves accountability in local cleanup activities, and reduces dependence on paper-based reporting. Its digital reporting and tracking workflow enables staff to identify locations with repeated complaints and prioritize follow-up actions. The system has also been assessed in terms of its operational requirements, including smartphone availability, camera and GPS permissions, mobile connectivity, and the accuracy of ward-boundary data. Accessibility limitations for residents without access to these resources have been identified, with alternative reporting channels recognized as part of the project's deployment considerations. Furthermore, the hotspot detection mechanism uses complaint-count thresholds as operational indicators rather than measures of actual disease incidence, maintaining a clear distinction between reported environmental concerns and confirmed public-health conditions.

CO2 response: Functional and non-functional requirements have been systematically separated and mapped to stakeholder needs and evidence from the current implementation. The requirements traceability assessment establishes the extent to which the implemented system satisfies the specified requirements and identifies the remaining gaps between the original SRS and the delivered application.

5. System Design (Design Session)

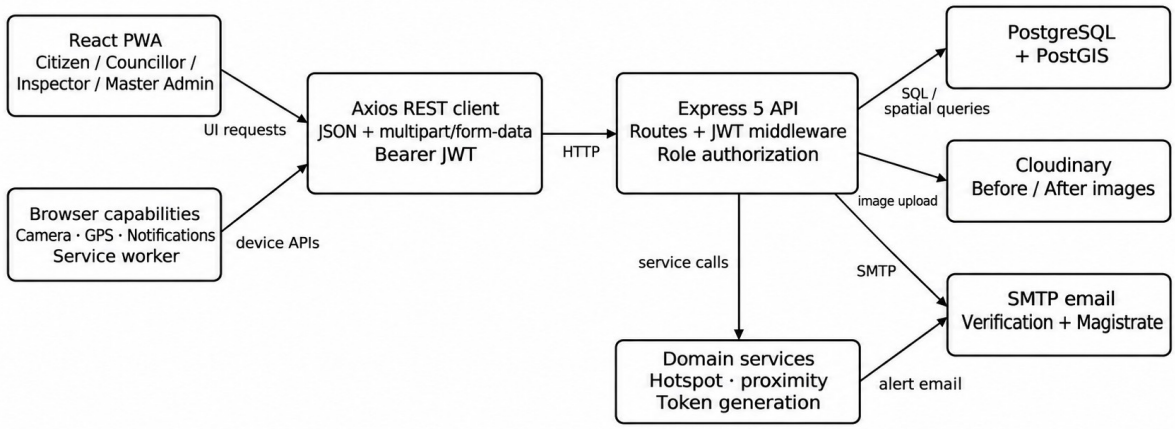
5.1 Design Evidence Note

The following architecture and data model are a **retrospective reconstruction from the latest frontend and backend code**, checked against the SRS. They are reconciled with the actual Design Document and design-session feedback.[10], [11]

5.2 Selected Architecture

The implemented system is a modular client-server web application:

- **Frontend:** React (version 18) PWA with React Router, reusable UI components, context-based authentication/language state, and Axios.
- **Backend:** Node.js with Express (version 5), REST endpoints, JWT authentication middleware, role-authorization middleware, and service modules.
- **Database:** PostgreSQL accessed through `pg`, with PostGIS spatial functions for ward and distance queries.
- **Image Storage:** Cloudinary stores before/after photos; the database stores image URLs.
- **Email:** Nodemailer SMTP sends citizen verification and Magistrate hotspot messages.
- **Browser Capabilities:** `getUserMedia` for camera capture, Geolocation API for GPS, Notification API and service worker for browser alerts, and service-worker caching for the PWA shell.



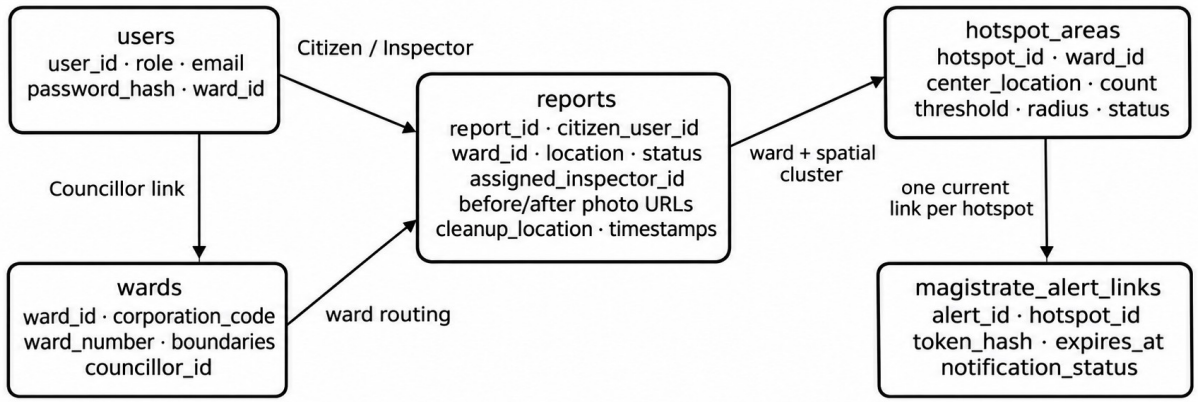
Architecture diagram

5.3 Component Responsibilities

Component	Responsibility	Main source files
Public and citizen interface	Landing page, registration, login, report capture, citizen report list.	LandingPage.jsx, Login.js, Register.js, ReportIssue.jsx, Dashboard.js
Councillor Interface	Ward report queue, filters, assignment, rejection, before/after verification.	CouncillorDashboard.jsx, ComplaintDetails.jsx
Inspector Interface	Assigned task list and cleanup proof capture.	InspectorDashboard.jsx, InspectorComplaintPage.jsx, CleanupUpload.jsx
Master Admin Interface	Staff creation/deactivation, report overview, hotspot list,	MasterAdminDashboard.jsx

	manual Magistrate alert resend.	
Authentication and Access Control	JWT storage/transport, role-aware routes, active-account check, server authorization.	AuthContext.js, authmiddleware.js, rolemiddleware.js
Spatial and Hotspot Services	Ward lookup, hotspot count/update/create, nearby-hotspot query.	reportRoutes.js, hotspotService.js, proximityAlertService.js
Magistrate Escalation	Secure token generation, token hashing, SMTP delivery, read-only report lookup.	magistrateAlertService.js, magistrateAlertRoutes.js, MagistrateHotspotDetails.jsx

5.4 Logical Data Model



Logical data model

Tables Used in Database	Key fields referenced by the backend	Purpose
users	user_id, full_name, email, role, password_hash, ward_id, is_active, email_verified, verification-token fields	Citizen and staff identity, role, account state, and authentication.
wards	ward_id, corporation_code, ward_number, boundaries, councillor_id	Ward boundary and Councillor assignment.
reports	report_id, citizen_user_id, description, location, ward_id, before_photo_url, after_photo_url, status, assigned_inspector_id, cleanup_location, cleanup_submitted_at, inspector_comment, councillor_note, timestamps	Main complaint, assignment, proof, and current status record.
hotspot_areas	hotspot_id, ward_id, center_location, complaint_count, threshold_count, detection_radius_meters, status,	Active complaint clusters.

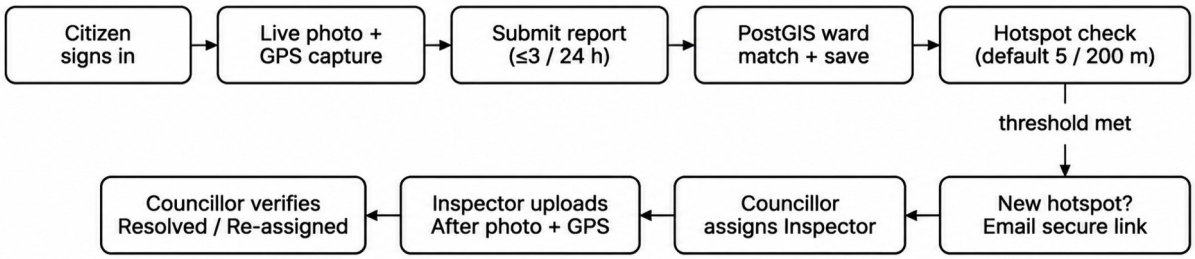
	timestamps	
magistrate_alert_links	alert_id, hotspot_id, token_hash, expires_at, notification_status, delivery_channel, last_error, sent_at, attempt_count	Current secure email link and delivery state for a hotspot.

The code implies these relationships: one Citizen can submit many reports; a Ward can contain many reports and hotspots; a Ward is linked to its Councillor; an Inspector can be assigned many reports; and a hotspot has a current alert-link record. The backend uses an upsert on `hotspot_id`, which replaces the token and expiry when an alert is reissued. Although the SRS says Cloudinary URLs and public IDs are stored, the observed report SQL persists image URLs only; the Cloudinary public ID is used for cleanup after a failed transaction and is not stored as a persistent report field.

5.5 Spatial and Workflow Design

Ward Routing: The backend constructs a point from longitude and latitude and runs `ST_Covers(boundaries, point)` against `wards`. A matching ward ID is stored with the report. If no supported ward contains the point, the request returns HTTP 422 and is not saved for later Master Admin review.

Hotspot detection: The default threshold is five complaints within 200 metres, evaluated within a ward. A new hotspot uses the triggering report's coordinates as its centre. An advisory transaction lock is used to serialize checks within a ward. When a new hotspot is created, the system attempts to send a Magistrate email after the report transaction commits. A notification failure does not roll back the saved report. Existing hotspots are updated, but the automatic email trigger is tied to new hotspot creation; Master Admin can manually resend an alert.



Actual workflow

5.6 Design Alternatives and Rationale

Design decision	Selected approach	Plausible alternative	Rationale and trade-off
Client Platform	React PWA	Native Android/iOS applications	A PWA reduces installation friction and supports one browser-based codebase. Native apps offer stronger control over background location and offline storage but require additional development and maintenance. Browser capabilities vary by device.
Backend Architecture	One Express	Separate	A modular monolith is simpler

	application divided into route and service modules	microservices	to develop, test, and deploy within a course project. Microservices could scale independently but add network, deployment, monitoring, and operational complexity.
Spatial Database	PostgreSQL + PostGIS	A general-purpose NoSQL database with external geospatial services	PostGIS supports relational joins, transactions, point-in-polygon matching, and metre-based distance queries in the same database. It requires a correctly configured extension and valid ward geometry.
Image Storage	Cloudinary	Self-hosted object storage or another cloud-storage service	Cloudinary avoids maintaining an image server and is consistent with the SRS's recorded cost-driven change. It introduces third-party availability, quota, privacy, and pricing dependencies.

5.7 Impacts and Design Justification (CO3)

- **Stakeholder Needs:** Role-scoped queues and ward routing reduce manual handoff; before/after evidence supports accountability.
- **Usability:** A browser-first PWA lowers installation barriers; camera and location permissions remain device-dependent.
- **Scalability:** PostGIS and relational queries provide a sound basis for ward-level spatial operations. Large-scale use still requires indexes, query plans, load tests, pagination, and operational monitoring.
- **Resource Use and Lifecycle Cost:** The modular monolith and managed image service reduce infrastructure work for a student prototype. Free-tier quotas and provider pricing must be checked before deployment.
- **Public Health and Safety:** The app supports environmental reporting, not diagnosis. A complaint-count threshold can be biased by reporting activity and must be validated before being used for official risk decisions.
- **Societal and environmental impact:** Better reporting may support source reduction and cleaner surroundings. Digital-only access, inaccurate GPS, false reports, and unequal smartphone access are potential exclusion or quality risks.

5.8 Design-Session Feedback

The supplied System Design Report documents the project's object-oriented design approach, UML class diagrams, and three-layer architecture.[11] However, some design descriptions reflect earlier choices, including Super Admin, Worker, SMS notifications, and Firebase Cloud Storage. These have been reconciled with the final implementation, which uses Master Admin, Inspector, email-based Magistrate escalation, and Cloudinary storage. The final architecture and data model are therefore presented in accordance with the latest working application.

CO3 response. The submitted design document provides the foundation for evaluating the system architecture and logical data model against the SRS and final implementation. The differences between the proposed design and implemented system are identified and addressed in this report

6. Implementation (Implementation Session)

6.1 Project Structure

The two archives contain separate frontend and backend projects. The following tree shows the principal source modules; build output, package-lock files, environment files, and Git internals are omitted for readability.

```
frontend/
  public/
  src/
    App.js
    axios.js
    index.js
    context/
      AuthContext.js
    i18n/
      LanguageProvider.jsx
      bn.json
      language.css
    components/
      BeforeAfterComparison.jsx
      CleanupUpload.jsx
      ConfirmDialog.jsx
      GoogleIdentityButton.jsx
      HotspotProximityMonitor.jsx
      InspectorAssignmentCard.jsx
      LiveCameraCapture.jsx
      MapLocationView.jsx
      Navbar.jsx
      PasswordVisibilityToggle.jsx
      StatusBadge.jsx
    pages/
      LandingPage.jsx
      Dashboard.js
      ReportIssue.jsx
      MagistrateHotspotDetails.jsx
      admin/MasterAdminDashboard.jsx
      auth/Login.js
      auth/Register.js
      auth/VerifyEmail.jsx
      councillor/CouncillorDashboard.jsx
      councillor/ComplaintDetails.jsx
      inspector/InspectorDashboard.jsx
      inspector/InspectorComplaintPage.jsx
    service-worker.js
    serviceWorkerRegistration.js
  package.json

backend/
  config/
    db.js
    cloudinary.js
  middleware/
    authmiddleware.js
    rolemiddleware.js
    upload.js
  routes/
    userRoutes.js
    reportRoutes.js
    inspectorRoutes.js
    hotspotRoutes.js
    magistrateAlertRoutes.js
  services/
    hotspotService.js
    magistrateAlertService.js
    proximityAlertService.js
    passwordPolicy.js
    verificationMail.js
  dhaka_wards.geojson
  server.js
  seedwards.js
  package.json
  README.md
```

6.2 Frontend-to-Backend Communication

The frontend uses Axios with a base URL of `/api` unless `REACT_APP_API_BASE_URL` is configured. An interceptor reads the JWT from browser `localStorage` and sends it as a Bearer token. Requests without a file normally exchange JSON; citizen and Inspector photo submissions use `multipart/form-data`.

API endpoint	Access	Main request / response purpose
POST /api/users/register	Public	Creates a Citizen account and sends a verification email.
POST /api/users/verify-email	Public	Verifies a 24-hour email token.
POST /api/users/resend-verification	Public	Resends a pending verification link, with a 60-second resend interval.
POST /api/users/login	Public	Checks email/password and returns JWT plus user/role/ward information.
POST /api/users/google-auth	Public	Verifies Google ID token; restricted to Citizen accounts.
GET /api/reports	Authenticated	Returns reports according to role: all for Master Admin, own ward for Councillor, assigned reports for Inspector, own reports for Citizen.
POST /api/reports	Citizen	Accepts description, latitude, longitude, and before_photo; also receives issue_type, which is not persisted by the current insert query.
GET /api/reports/councillor	Councillor / Master Admin	Returns ward reports for a Councillor or all eligible reports for Master Admin.
GET /api/reports/:id	Councillor / Inspector / Master Admin	Returns a report if the user satisfies the role and ownership/ward scope.
PATCH /api/reports/:id/assign	Councillor / Master Admin	Accepts inspector_id; changes report to Assigned.
PATCH /api/reports/:id	Inspector	Accepts after_photo, latitude, longitude, and optional inspector_comment; changes status to Pending Verification.
PATCH /api/reports/:id/verify	Councillor / Master Admin	Accepts is_approved; sets Resolved or Re-assigned.
PATCH /api/reports/:id/reject	Councillor	Accepts a categorized rejection reason; sets Rejected for eligible unassigned reports.
GET /api/inspectors	Councillor / Master Admin	Lists active Inspectors in the relevant ward or across the system.
GET /api/users/staff	Master Admin	Returns Councillor and Inspector directory data.
POST /api/users/create-staff	Master Admin	Creates a Councillor or Inspector for an existing ward.
DELETE /api/users/staff/:userId	Master Admin	Deactivates staff; active Inspector assignments are returned to the reassignment queue.
GET /api/hotspots	Public	Returns active hotspot areas.
GET /api/hotspots/nearby	Public	Accepts latitude/longitude and returns active hotspots within the configured radius.
POST /api/hotspots/:id/magistrate-alert	Master Admin	Reissues the 48-hour Magistrate email link.

GET /api/magistrate-notify/:token	Public token link	Returns a read-only hotspot summary and up to 100 nearby complaints if the token is valid and email delivery succeeded.
GET /api/test-db	Public diagnostic	Returns a database connection check.

6.3 Data Exchanged

Authentication Payload. Email/password login returns a JWT and a user object containing `user_id`, `full_name`, `email`, `role`, and `ward_id`. The frontend stores the token and user object in `localStorage`. Google sign-in is verified again by the backend.

Citizen Report Payload. The browser sends a multipart form with a live image, latitude, longitude, description, and issue type. The backend validates the authenticated account, checks the rolling quota, validates coordinate ranges, finds a ward, uploads the image to Cloudinary, and inserts the report. The current SQL insert stores the description, location, ward, before-photo URL, status, and timestamps. It does not store `issue_type`.

Cleanup Payload. The Inspector sends an after-photo, GPS coordinates, and optional comment. The backend checks that the report is assigned to the logged-in Inspector and is still in `Assigned` state. It uploads the image to Cloudinary, stores the cleanup point and submission timestamp, and changes the status to `Pending Verification`.

Verification Payload. The Councillor sends an approval boolean. The backend sets `Resolved` when approved and `Re-assigned` when rejected for further cleanup. The separate rejection endpoint sets `Rejected` with a reason for an eligible unassigned complaint.

6.4 Important Implementation Algorithms

1. **Ward Lookup:** the backend uses `ST_Covers` with the report's GPS point and ward polygon. The query uses parameterized values and selects one matching ward.
2. **Rolling Complaint Limit:** the backend checks reports created in the previous 24 hours. It locks the user row inside a transaction before the authoritative count, reducing the chance that simultaneous submissions bypass the limit. Rejected reports remain part of the count.
3. **Hotspot Detection:** the backend checks complaints within the configured distance and ward. A transaction-level advisory lock serializes hotspot checks for that ward. The default minimum is five complaints and the default detection radius is 200 metres.
4. **Email Link Security:** the backend generates a 32-byte random token, stores its SHA-256 hash, and sets an expiry 48 hours in the future. Reissuing the alert replaces the current token hash and expiry for that hotspot.
5. **Image Cleanup on Failed Report Creation:** if an image has been uploaded but the database transaction fails, the backend attempts to delete the unused Cloudinary object.

6.5 Implemented Features

- Public landing page with navigation to login and registration.
- Citizen email/password registration with email verification.
- Google sign-in for Citizen accounts.
- JWT-based authentication and server-side role authorization.
- Master Admin staff creation and deactivation.
- Citizen live camera capture with GPS.
- Three-report rolling 24-hour limit.
- Automatic ward routing through PostGIS.
- Citizen report tracking by current status.
- Councillor ward dashboard, report detail, Inspector assignment, rejection, and verification.
- Inspector task list and after-cleanup photo/GPS upload.
- Before/after image comparison.
- Active hotspot detection and public hotspot lookup.

- Secure Magistrate email alert link with 48-hour expiry.
- Browser proximity alerts for opted-in users.
- Bangla/English interface support.
- PWA app-shell caching and image caching.

6.6 Code Organization and Rationale

The frontend uses functional React components rather than class-based components. Pages represent role-specific workflows; smaller components handle camera capture, cleanup upload, status badges, map location, before/after comparison, and Inspector selection. `AuthContext` centralizes user/session state, while `LanguageProvider` manages the language preference and UI translation. `axios.js` centralizes API configuration and Bearer-token attachment.

The backend separates HTTP routes from cross-cutting middleware and domain services. Authentication and role checks are middleware; spatial hotspot evaluation and proximity checks are service functions; database and Cloudinary configuration are isolated in `config/`. This structure keeps the main application modular, although SQL and orchestration remain concentrated in large route files such as `reportRoutes.js` and `userRoutes.js`.

6.7 Function and Class Size Analysis

The count excludes blank lines and comment-only lines but includes JSX lines inside component bodies. Anonymous inline callbacks are not counted as separate named units. These figures are source-structure metrics, not a measure of code quality.

Codebase	Named function/component units	Maximum non-empty LOC	Largest unit	Average non-empty LOC
Frontend	104	476	ComplaintDetails (src/pages/councillor/ComplaintDetails.jsx)	45.6
Backend	22	168	evaluateHotspotForReport (services/hotspotService.js)	25.5
Combined	126	476	ComplaintDetails	42.1

Class-level statistics: no JavaScript `class` declarations or class methods were used. The project uses functions and modules rather than class-based object-oriented design.

6.8 Implementation-Session Feedback

No implementation-session review sheet, presentation, or feedback minutes were included. The current source code archive is used as the final implementation baseline. If the team has session feedback, add the comment, the affected module, the resulting change, and the related commit or screenshot.

CO1 and CO3 response: The code demonstrates programming, data structures, relational queries, spatial algorithms, transactions, middleware, and component-based design. The actual implementation largely follows a modular client-server design, while several planned SRS components—such as a separate status-history model and offline queue—were not implemented.

7. Tools, Version Control, Testing, and CI/CD

7.1 Frameworks, Libraries, and Tools

Layer / purpose	Technology observed	Reason for use and limitation
Frontend Framework	React 18.3.1, React DOM 18.3.1	Component-based UI and reusable role-specific pages. Requires a JavaScript-capable browser and careful state management.
Routing	React Router 6.26.0	Separates public and role-protected routes. Client-side route guards must be backed by server authorization, which is also present.
HTTP client	Axios 1.7.0	Centralized REST calls and Bearer-token interceptor. Token storage in localStorage requires strong XSS prevention.
Styling/Build	Tailwind CSS 3.4.1, PostCSS, Autoprefixer, Create React App scripts 5.0.1	Responsive styling and build pipeline. The supplied landing-page notes state that a production build could not be verified in that environment after npm ci timed out.
Backend Framework	Node.js / Express 5.2.1	Lightweight REST API and middleware pipeline. Validation, rate limiting, and security headers require deliberate configuration.
Database Driver	pg 8.23.0	PostgreSQL connection pool and parameterized queries.
Spatial Database	PostgreSQL + PostGIS	Point-in-polygon ward routing and metre-based proximity calculations. The database/server version and schema migrations were not supplied.
Image Upload/Storage	Multer 2.2.0 + Cloudinary 2.11.0	Receives images in memory and uploads them to managed image storage. File size/content validation needs strengthening.
Authentication/Security	bcrypt 6.0.0, jsonwebtoken 9.0.3, google-auth-library 11.1.0	Password hashing, signed JWT sessions, and server-side Google ID-token verification.
Email	Nodemailer 10.1.2	SMTP email for verification and Magistrate alerts. Delivery depends on external SMTP configuration.
Configuration	dotenv 17.4.2, cors 2.8.6	Environment-based configuration and cross-origin access. Current CORS origins are hard-coded in server.js.
Maps	OpenStreetMap embed and Google Maps links	Complaint and hotspot location display. Mapbox is not used in the

		supplied ; Leaflet packages are declared.
Browser/PWA APIs	Geolocation, getUserMedia, Notification API, service worker	GPS, live camera, browser alerts, and app-shell caching. Background execution are implemented.
Development/Version Control	Visual Studio Code, Git, GitHub	Source editing and version history. The uploaded frontend archive has no Git metadata.

7.2 Plausible Tool Alternatives and Limitations (CO4)

- **React versus A server-rendered Express/EJS interface:** React supports reusable interactive pages and role-specific dashboards. A server-rendered interface could reduce client-side JavaScript but would make the current component interactions and client-side routing less direct.
- **Express versus Django:** Express is lightweight and aligns with the JavaScript frontend. Django provides more built-in administration, ORM, and security conventions, but would introduce a Python backend and a different integration model.
- **PostgreSQL/PostGIS versus Firebase/Firestore:** a managed NoSQL service can simplify real-time synchronization, but the current workflow needs relational joins and spatial operations such as `ST_Covers` and `ST_DWithin`. PostGIS is a better fit for the current ward-routing implementation, at the cost of database administration and spatial-data preparation.
- **Cloudinary versus Self-hosted image storage:** Cloudinary reduces the need to operate an image server and provides managed delivery. Self-hosted or object storage can offer more deployment control but requires storage configuration, access policies, backups, and image-delivery work.

7.3 Version-Control Evidence

The project was developed and maintained through Git and GitHub, with a commit history spanning **17 May to 28 October 2026**. The visible commit history records contributions under three GitHub accounts: **`sheikhirtiaj`**, **`rufayed-rehan`**, and **`adibaahsan04080`**. It also documents several merged pull requests (#1–#7), demonstrating the integration of work across development branches.[12]

The commit history reflects the project's progression through several major development milestones:

1. **Initial setup:** Node.js, Express, PostgreSQL connection, environment configuration, and initial backend structure. *(**Commits:** f5d2896, a2cb61f, 6b621b5, 5428589)*
2. **Authentication and authorization:** User registration, login, password hashing, JWT authentication, and role-based access control. *(**Commits:** afb9309, dd9ef9e, 5e1dbab, 429887b)*
3. **Core reporting and GIS:** Report submission, image attachment, PostGIS integration, ward identification, and location-based routing. *(**Commits:** 9637d55, 041c99c, 4ae1180, 9d1e06d, 11db8c9)*
4. **Administrative functionality:** Councillor and Inspector dashboards, staff management, ward-based access restrictions, and Inspector assignment. *(**Commits:** 5371a42, e3ae8dc, 167f614, 4395713, 33e5618, e1b8b34)*
5. **Evidence and verification:** Cloudinary integration, before-and-after image uploads, cleanup task completion, and Councillor verification. *(**Commits:** 0646c87, 7eb7825, 6457f97, 616872d, a3c9d97)*
6. **Hotspot management:** Complaint-cluster detection, hotspot records, Magistrate alert services, and proximity-based sound notifications. *(**Commits:** 9096841, 44444b3, eeb99db)*
7. **Frontend development:** Progressive Web App (PWA) support, dashboard enhancements, complaint management, and interface improvements. *(**Commits:** 135285d, 04865ef, 64d1f00, 9ef56c7, cf2ce4d, c064f96, 32318fd)*

7.4 Testing Approach and Evidence

The application underwent functional and workflow testing throughout development to verify the correctness and reliability of its core features. Tests were performed on both backend services and frontend interactions, with the observed outputs matching the expected results.

Testing covered the major application workflows, including:

1. **Authentication:** Registration, login, email verification, and role-based access control.
2. **Report Management:** Report submission, image uploads, GPS validation, and ward identification.
3. **Administrative Workflows:** Inspector assignment, ward-based access, cleanup proof submission, and Councillor verification.
4. **Hotspot Management:** Complaint-cluster detection, proximity alerts, and Magistrate notification.
5. **Error Handling:** Invalid inputs, unauthorized access, and unsuccessful operations.

7.5 CI/CD and Build Reproducibility

The project does not include a configured CI/CD pipeline or GitHub Actions workflow. Build, testing, and deployment processes are managed without automated pipeline integration.

8. Professional, Ethical, and Legal Considerations

8.1 Data Handled By The Application

The application processes:

- Names and email addresses;
- Password hashes and authentication tokens;
- User roles and ward assignments;
- Precise citizen report coordinates;
- Report descriptions and before/after photographs;
- Inspector cleanup coordinates and comments;
- Hotspot locations and complaint counts;
- Magistrate alert tokens and delivery metadata.

Such information may expose individuals' identities, routines, and whereabouts. Our App's deployment establish clear access permissions, retention periods, data correction and deletion procedures, and protocols for responding to data breaches.

8.2 Security Controls

Control	Evidence in codebase
Password Hashing	bcrypt.hash(..., 12) is used for password storage.
Password Policy	New passwords must be 8–128 characters and satisfy at least three of four character classes.
JWT Authentication	Bearer token verification is performed by authmiddleware.js; tokens are issued with a one-day expiry.
Role Authorization	rolemiddleware.js checks the exact allowed role for protected routes.
Active-account Enforcement	The authentication middleware checks account status on protected requests.
SQL Parameterization	SQL queries use positional parameters such as $1, $2, reducing SQL-injection risk.
Email Verification	Verification tokens are hashed before storage and expire after 24 hours.
Magistrate Link	A 32-byte random token is hashed at rest and expires after 48 hours. Reissuing the link replaces the current token.
Sensitive Page Headers	The verification and Magistrate routes set no-store/no-referrer/noindex headers.
Upload Authorization	Report upload requires Citizen; cleanup upload requires Inspector; assignment and verification are role-restricted.

8.3 Ethical Considerations

- **Report Accuracy and Verification:** Citizen submissions are treated as unverified reports until Councillor review. Councillors can verify, reject, or reassign reports. Automated duplicate detection is not implemented.
- **Photo Privacy:** The reporting interface advises citizens to avoid capturing identifiable bystanders unless necessary. Sensitive images require appropriate handling and removal procedures.

- **Reporting Accessibility:** Report submission relies on a smartphone, camera, GPS, internet connection, and location permission. Alternative reporting channels are needed for residents without these resources.
- **Language Inclusion:** The interface supports Bangla and English. Translation completeness and accessibility require validation with actual users.
- **Inspector Safety:** The application records cleanup locations and before-and-after photo evidence. It does not include a complete safety checklist, work-start confirmation, or incident-reporting feature.
- **Responsible Hotspot Alerts:** Hotspot detection uses configured complaint-cluster thresholds and proximity warnings. These require appropriate validation to minimize false alarms and avoid misleading interpretations.

8.4 Licensing and Legal Review

Licensing and legal considerations were taken into account throughout the project. The backend package specifies the ISC license, while the licensing terms of frontend packages, third-party dependencies, visual assets, and ward GeoJSON data are recognized as important considerations for responsible software distribution. The project also considers applicable Bangladesh requirements concerning data protection, cybersecurity, public health, local-government operations, and records retention, particularly in handling citizen information, location data, and uploaded images.

CO4 response: The application incorporates security controls and considers the legal, ethical, and professional responsibilities associated with handling public reports and personal information. Security, privacy, licensing, and regulatory requirements remain important considerations for maintaining the system and preparing it for real-world deployment.

9. Project Management and Feasibility

9.1 Planned Timeline in The SRS

The SRS includes a proposed W3–W15 timeline (SRS, p. 18) [10] covering environment/database setup, backend API construction, React PWA UI, integration, dashboard preparation, media storage, spatial mapping, cleanup verification, hotspot email escalation, proximity alerts, quality assurance, and final presentation. It was a planned timeline, not a dated record of actual completion.

9.2 Development Chronology Evidenced by Git History

Date / period	Commit-history evidence	What it indicates
17–18 May 2026	`f5d2896` – Initial commit; `a2cb61f` – Node.js and Express setup; `6b621b5` – Database connection pool and environment variables; `fa273d9` – Database pool fix; `5428589` – PostgreSQL connection; `041c99c` – Report routes; `9637d55` – Report API.	Initial backend architecture, database connectivity, and report-management foundation.
19–21 May 2026	`a7bb2c4` – PATCH functionality; `4ae1180` – Report submission API; `afb9309` – Registration, bcrypt, and JWT implementation.	Report creation, modification, and authentication functionality.
27–30 May 2026	`dd9ef9e` – Login implementation; `2bcae96` – PATCH request validation; `5e1dbab` – Role checker.	Authentication and role-based access control.
5–16 June 2026	`429887b` – Assigned Inspector login verification; `9d1e06d` – PostGIS integration; `a888c9c` – Location update path; `11db8c9` – Ward data and 40-ward configuration; `4e7eb8e` – Citizen form and frontend configuration updates; `99c41f4` – Removal of `.env` from Git tracking.	Spatial database integration, ward configuration, location handling, and repository configuration.
14–19 July 2026	`8ceee28`, `969c7fe` – Frontend repository organization; `cf3b957` – Phone/email registration; `88fc4c8` – Bug fixes.	Frontend branch organization and registration refinements.
2 August 2026	Cloudinary image storage and upload fixes.`32318fd` – StatusBadge, ConfirmDialog, and Navbar; `a3c9d97` – MapLocationView and BeforeAfterComparison; `c2fcb90` – InspectorAssignmentCard and CleanupUpload; `e3ae8dc` – CouncillorDashboard and ComplaintDetails; `5371a42` – Inspector pages and role-based routing; `5c3029b` – Dashboard Hooks fix; `ec11460` – Merge PR #1.	Development of role-specific dashboards, complaint-management interfaces, reusable components, and frontend routing.
21 August 2026	`4395713` – Staff information and Master Admin API; `167f614` – Inspector lookup and selection; `5ce911f` – Frontend localhost configuration.	Staff management and Inspector assignment support.
2 October 2026	`0646c87` – Cloudinary image storage integration; `7eb7825` – Image-upload API correction.	Cloud-based image storage and reliable photo-upload functionality.
13–17 October 2026	`6457f97`, `2a22641`, `33e5618`, `9096841` (commit ID's)	Core reporting workflow, ward-based access control, Master Admin management, and hotspot detection.

22–27 October 2026	`c064f96`, `616872d`, `44444b3`, `135285d` (commit ID's)	Dashboard improvements, cleanup workflow refinement, backend enhancements, and PWA integration.
28 October 2026	`eeb99db`, `3d18165`, `410c365` (Commits ID's)	Nearby hotspot sound alerts, final refinements, and pull-request integration.

9.3 Planned Versus Evidenced Implementation

Planned area	Planned Functionality	Implemented Functionality
Environment and Database	Backend environment, database connectivity, and spatial data support.	Express server, PostgreSQL connection pool, and PostGIS queries.
Backend API	REST APIs for users, reports, Inspectors, hotspots, and Magistrate alerts.	Implemented API routes covering all major application modules.
Frontend UI	Role-based interfaces for Citizens, Councillors, Inspectors, Master Admin, and Magistrate.	React-based dashboards and pages for all five user roles.
Frontend–Backend Integration	Connect frontend components with backend services and support data submission.	Axios REST integration with JSON requests and multipart file uploads.
Cloud Image Storage	Store report and cleanup photographs using cloud storage.	Cloudinary integration for image uploads and storage.
Spatial Mapping and Routing	Identify wards from report coordinates and perform location-based hotspot calculations.	PostGIS-based ward identification and hotspot distance queries.
Cleanup verification	Inspector assignment, field evidence submission, and Councillor verification.	Inspector proof uploads with Councillor approval, rejection, and reassignment.
Email Escalation	Notify the Magistrate about unresolved hotspot cases.	Time-limited 48-hour secure alert links with manual resend functionality.
Proximity Alerts	Notify users about nearby dengue hotspots.	Partially implemented Nearby-hotspot API, browser permission handling, and sound alerts.
Offline Support	Support report submission during temporary network interruptions.	PWA service-worker caching for static assets; offline report queuing is not included.
Testing and Refinement	Validate application workflows and improve functionality through testing.	Functional testing, expected-output verification, bug fixing, and interface refinements.
Version Control and Integration	Maintain source code and coordinate development across team members.	Git/GitHub version control, feature branches, and pull-request integration.
CI/CD and Deployment	Automate application build, testing, and deployment.	Application deployment and GitHub-based development integration; automated CI/CD pipeline is not configured.

9.4 Scope Changes and Prioritization

The SRS documents four key changes to the original proposal: email-based registration, Cloudinary integration in place of Firebase Cloud Storage, email-based Magistrate escalation instead of SMS, and

a limit of three report submissions per 24 hours.[10] These decisions shaped the final implementation by simplifying communication and storage workflows while introducing a basic measure against excessive submissions. Google sign-in was also incorporated for Citizens.

During development, the team prioritized the application's core operational workflow, covering report submission, ward identification, Inspector assignment, cleanup evidence collection, Councillor verification, and hotspot alerts. Other proposed capabilities, including duplicate grouping, offline report queuing, audit history, and administrative ward and threshold configuration, remained outside the delivered implementation scope.

9.5 Infrastructure and Feasibility

The application requires the following technical environment:

- **Backend and Database:** Node.js, PostgreSQL with PostGIS, and valid ward polygon data.
- **Frontend:** Installed dependencies and a production build.
- **External Services:** Cloudinary, SMTP, and Google Web Client ID credentials.
- **Configuration:** JWT secret, public URLs, hotspot thresholds, and radius settings.
- **Deployment:** HTTPS hosting with an approved CORS origin.

The application is feasible using a lightweight web architecture. Production deployment requires finalized database setup, ward data, documentation, testing, and hosting configuration.

9.6 Effort and Third-Party Cost Estimates

Cost / effort item	Current evidence	Required estimate before deployment
Development Effort	Git history spans May–October; no timesheets or task-board export.	Person-hours by frontend, backend, GIS/data preparation, testing, documentation, and deployment.
Database Hosting	PostgreSQL/PostGIS required; no provider/instance selected in the supplied files.	Instance, storage, backup, and bandwidth cost.
Image Storage	Cloudinary integration; no usage forecast or subscription record.	Expected number and size of before/after images and provider quota/overage.
Email	SMTP/Nodemailer; no provider plan or volume record.	Verification and hotspot email volume and any provider charges.
Web Hosting/Domain	No deployment manifest or provider specified.	Hosting, domain, TLS, monitoring, and backup cost.
Map Service	OpenStreetMap embed and Google Maps links in the UI.	Review provider terms, attribution, usage limits, and any production API costs.

CO7 response: The backend history demonstrates iterative development across several months and identifies major milestones. The available evidence does not establish a full planned-versus-actual project schedule, team task tracking, numeric effort estimates, or provider costs. The architecture is feasible for a course app, but production feasibility requires the missing operational and cost evidence.

10. Teamwork and Individual Contributions

10.1 Team Roster

Team member	Student ID	Responsibility evidence in supplied archive
Md Rufayed Ul Alam	230042121	Frontend Development: Developed Councillor and Inspector dashboards, complaint management pages, registration UI, dashboard widgets, cleanup upload components and map views.
Sheikh Irtiaj Ahmed	230042130	Backend Development: Developed authentication, REST APIs, PostgreSQL/PostGIS integration, ward-based access control, Inspector assignment, Cloudinary uploads, cleanup verification, Master Admin functionality, and hotspot services.
Adiba Ahsan Raiyan	230042161	Frontend Development: Worked on frontend project setup, registration configuration, PWA support, frontend–backend integration and nearby-hotspot sound notifications.

10.2 Interdependent Modules and Integration Points

The implementation contains several cross-module dependencies that the team needed to coordinate:

1. **Authentication Contract:** the frontend expects a JWT and user object containing the same role strings used by backend middleware. A role-name mismatch can prevent correct dashboard routing.
2. **Report Contract:** the camera component must provide a file and GPS coordinates; the frontend sends multipart form data; the backend validates it and returns the report row.
3. **Ward Contract:** the database ward IDs and polygon data must match the staff ward assignments and the frontend's displayed ward information.
4. **Assignment Contract:** the Councillor selects an Inspector; the backend checks the Inspector's role, active status, ward, and the Councillor's authority before assignment.
5. **Proof Contract:** the Inspector sends an after-photo and cleanup GPS; the Councillor screen expects the resulting `Pending Verification` state and the two image URLs.
6. **Hotspot Contract:** the backend returns active hotspot data and a nearby radius; the frontend uses the data for the proximity panel; the Magistrate page expects the token endpoint's read-only response shape.

10.3 Disagreements, Blockers, and Inclusion

No records of disagreements, blockers, task reassignment, meeting attendance, or inclusion practices were done.

CO5 response: The GitHub commit history and merged pull requests demonstrate active collaboration among all three team members across frontend and backend development. Each member contributed to identifiable features, functional improvements, and project integration. The recorded development activities provide evidence of individual responsibilities and collaborative work throughout the project.

11. Results, Limitations, and Future Work

11.1 Final Implementation Status

Capability	Status	Explanation
Public Landing Page	Completed	Public overview is available before login.
Citizen Registration and Sign-In	Partial	Email verification and Google sign-in are present; password reset is absent.
Four Application Roles	Completed	Citizen, Inspector, Councillor, and Master Admin are enforced in the supplied code.
Live Report Photo and GPS	Completed	Live capture is implemented; description is required.
Rolling 3-per-24-Hour Limit	Completed	Enforced by the backend.
Automatic Ward Routing	Completed	PostGIS matching is implemented; no-match reports are rejected rather than queued for review.
Councillor Ward Scope	Completed	Backend filters reports by assigned Councillor/ward.
Inspector Assignment	Completed	Master Admin/Councillor can assign eligible Inspectors.
Cleanup Proof Upload	Partial	After photo, GPS, and comment are stored; no Start Work state or offline queue.
Councillor Verification/Reassignment	Completed	Resolved or Re-assigned status is written.
Rejection with Reason	Completed	Eligible reports can be rejected with a categorized reason.
Hotspot Detection	Completed	Default five complaints or more within 200 metres, evaluated by ward.
Magistrate Secure Link	Completed	Hashed token and 48-hour expiry; one configured email and manual retry.
Proximity Alerts	Partial	Browser opt-in and location monitoring; not a background push service.
Bangla/English	Partial	Language provider and dictionary exist; full coverage unverified.
Offline Submission Queue	Not implemented	Service worker caches assets, not report/action submissions.
Duplicate Grouping and Priority Sorting	Not implemented	No corresponding algorithm found.
Ward/Zone/threshold Admin UI	Not implemented	Thresholds are environment-configured; ward data is seeded separately.
Status History and Audit Log	Not implemented	Current status is stored; separate history/audit records were not found.
Password Reset/Profile update	Not implemented	No corresponding endpoint/page found.
Inspector Reminder/History	Not implemented	Only active assigned and pending-verification work is shown.
Automated Tests and CI/CD	Not implemented	No test files, test reports, or workflow configuration were supplied.

11.2 Known Limitations

1. **Offline Functionality:** The application supports PWA caching but does not queue report submissions or Inspector actions when offline.
2. **Workflow Management:** The current workflow does not include a separate Start Work or In Progress state, automated report-status notifications, or delayed-task reminders.
3. **Report Management:** Automated duplicate detection, duplicate grouping, and priority-based report sorting are not implemented.
4. **Administrative Control:** Ward and zone editing, hotspot threshold configuration, per-zone Magistrate email settings, and audit logging are not available through the application.
5. **Security Enhancements:** File uploads require stronger size and content validation, while general API rate limiting and automatic email retries remain areas for improvement.[13]
6. **Deployment and Documentation:** Database schema files, migrations, and ward GeoJSON data are not included in the supplied archive, limiting reproducibility of the deployment environment.

11.3 Prioritized Future Work

Priority 1 — Deployment Readiness

- Finalize database migrations, ward GeoJSON, and deployment documentation.
- Secure environment variables and rotate exposed credentials.
- Organize the repository and introduce automated testing and CI.

Priority 2 — Functional Enhancements

- Implement offline reporting and synchronization.
- Introduce task-start tracking, work history, reminders, and status notifications.
- Add duplicate detection, priority sorting, and audit logs.
- Expand Master Admin controls for ward, zone, and hotspot configuration.

Priority 3 — Security and Scalability

- Strengthen file-upload validation and API rate limiting.
- Improve privacy controls, data retention, and secure deployment.
- Conduct cross-device, browser, accessibility, and performance testing.
- Validate hotspot thresholds with public-health authorities and introduce monitoring and backup procedures.

11.4 Overall Outcome and Communication

The implemented application delivers an end-to-end workflow, from citizen reporting to verified cleanup, supported by complaint-cluster detection and secure Magistrate alerts. The project establishes a functional foundation for dengue-hazard reporting and response coordination, with further scope for operational refinement, testing, and deployment improvements.

This report presents the project's objectives, user roles, system architecture, data flow, implementation, and future development in a structured manner. It consistently follows the terminology of the final implementation and explains key technical concepts through the glossary.

CO6 response. The project outcomes are communicated through the executive summary and detailed technical documentation. The Bangla/English interface further supports communication with users from different language backgrounds.

12. Glossary and References

12.1 Glossary

Term	Meaning
API	Application Programming Interface; a defined way for the frontend and backend to exchange requests and responses.
Bearer token	A token sent with a request; possession of the token grants the permissions associated with it.
Cloudinary	Managed cloud service used to store and deliver report images.
Councillor	Ward-level authority who reviews reports, assigns Inspectors, and verifies cleanup evidence.
GeoJSON	JSON-based format for representing geographic features such as points and ward polygons.
Geofencing	Checking whether a coordinate is inside a geographic boundary or within a specified distance of a location.
GIS	Geographic Information System; tools and methods for storing, analysing, and displaying geographic data.
GPS	Global Positioning System; provides device location coordinates.
Hotspot	In this project, a location cluster meeting the configured complaint-count and distance threshold; not a clinical diagnosis.
Inspector	Field-response staff member assigned to perform cleanup and upload evidence.
JWT	JSON Web Token; a signed token used to represent an authenticated session.
Master Admin	Highest application role in the actual implementation; manages staff and system-level oversight.
PostGIS	PostgreSQL extension for storing and querying spatial data.
PWA	Progressive Web App; a browser application that can use service-worker and installable-app capabilities.
RBAC	Role-Based Access Control; permissions are determined by the authenticated user's role.
REST	Representational State Transfer; a common style for HTTP APIs.
SMTP	Simple Mail Transfer Protocol; used by the application to send email.
ST_Covers	PostGIS function that tests whether one geometry covers another, including its boundary.
ST_DWithin	PostGIS function that tests whether two geometries are within a given distance.
Transaction	A group of database operations committed together or rolled back on failure.
Ward	Administrative area used to route a report to the responsible Councillor.

12.2 References

[1] World Health Organization, “Engaging communities to sustain dengue vector control.” Available at: https://www.who.int/activities/engaging-communities-to-sustain-dengue-vector-control/engaging-communities-to-sustain-dengue-vector-control (accessed 31 October 2026).

[2] World Health Organization, “WHO launches global strategic plan to fight rising dengue and other Aedes-borne arboviral diseases,” 3 October 2024. Available at: https://www.who.int/news/item/03-10-2024-who-launches-global-strategic-plan-to-fight-rising-dengue-and-other-aedes-borne-arboviral-diseases (accessed 31 October 2026).

[3] World Health Organization, *Risk communication and community engagement readiness and response toolkit: dengue fever*, 20 June 2024. Available at: https://www.who.int/publications/i/item/9789240095274 (accessed 31 October 2026).

[4] Open311, “GeoReport v2.” Available at: https://wiki.open311.org/GeoReport_v2/ (accessed 31 October 2026).

[5] Ushahidi, “Features.” Available at: https://www.ushahidi.com/features/ (accessed 31 October 2026).

[6] KoboToolbox, “Collecting GPS data with KoboToolbox,” last updated 23 April 2026. Available at: https://support.kobotoolbox.org/collect_gps.html (accessed 31 October 2026).

[7] DHIS2 Documentation, “Tracker implementation: Introduction.” Available at: https://docs.dhis2.org/en/implement/tracker-implementation/introduction.html (accessed 31 October 2026).

[8] PostGIS Project, “ST_Covers.” Available at: https://postgis.net/docs/ST_Covers.html (accessed 31 October 2026).

[9] PostGIS Project, “ST_DWithin.” Available at: https://postgis.net/docs/ST_DWithin.html (accessed 31 October 2026).

[10] Project Team (Group 14), *Software Requirements Specification (SRS): Dengue Alert & Ward-Based Management System*, SWE 4404: Software Project Lab II, Version 1.0, 2026.

[11] Project Team (Group 14), *System Design Report: Dengue Alert & Ward-Based Management System*, SWE 4404: Software Project Lab II, 2026.

[12] Project Team (Group 14), “SPL-II-23: Dengue Alert & Ward-Based Management System,” GitHub repository. Available at: https://github.com/sheikhirtiaj/SPL-II-23 (accessed 31 October 2026).

[13] OWASP Foundation, “File Upload Cheat Sheet,” OWASP Cheat Sheet Series. Available at: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html (accessed 31 October 2026).